

Plant Disease Classification (Mroot)

The goal of this project was to build a model that predicts plant diseases from images while ignoring the plant type. At first, I treated it like a normal classification task, but this approach did not work.

I started by training a model on the original dataset, which had around 82 classes combining plant and disease. The results were extremely low (around 2%), which showed that the model was not learning anything useful. After analyzing this, I realized that the problem setup itself was incorrect, because the task requires predicting only the disease.

I then reorganized the dataset into disease-only classes. At first I made a mistake and created around 40 classes, but later corrected it to 39 proper classes. This step had the biggest impact on performance. Once the labels were fixed, the model started to learn much better.

For the model, I used EfficientNet-B0. The training setup included image size 288x288, weighted sampling to handle class imbalance, MixUp augmentation, and AdamW optimizer. I also tried several other approaches, including ResNet34, stronger regularization, progressive resizing, and Vision Transformer. None of these improved the results compared to the main setup. In particular, Vision Transformer overfit quickly and was not stable for this dataset.

After reaching a baseline of around 0.87 mAP, I analyzed the model's mistakes. Most errors happened between visually similar diseases, such as different leaf spot types and rust variants. To improve this, I reduced MixUp from 0.2 to 0.1, which helped the model focus more on fine details. I also tried a more complex approach using grouped classes and multiple models, but it performed worse than a single model, so I did not use it.

During training, I observed results close to 0.89 mAP, but these were based on an earlier validation setup. After fixing the validation split and ensuring it was consistent, the final results became around 0.88 mAP on validation and around 0.86 mAP on the test set. These results are slightly lower but more reliable.

The main conclusion from this project is that defining the problem correctly is more important than trying many different models. Once the dataset was organized properly, a relatively simple model achieved good performance.